

OpenCode × iAgent/standard (DeepSeek V4 Flash 0731) 对齐 Claude Code

同一 DeepSeek V4 Flash 0731 模型下的 Harness workflow 差异、结果根因与源码
改造方案

workflow 差异 → 结果因果链 → 最小改造路径 → 评测闭环

本报告中的 iAgent/standard 是什么？

公司内部路由别名：iAgent/standard → DeepSeek V4 Flash 0731（公司本地部署）。这里的“0731”是本项目锁定的模型构建/版本标识。后文凡写“公司模型”均指这一固定服务实例。外部 DeepSeek V4 Flash 实验并无证据表明使用了同一 0731 build，只作为 same-model Harness A/B 的方法类比。

核心问题

如果 OpenCode 与 Claude Code 都驱动同一个公司本地 DeepSeek V4 Flash 0731，能否通过改造 OpenCode Harness，让公司实际 Coding workload 的 verified quality 达到 Claude Code 水平？如果能，差距来自哪一段 workflow，应该改哪几处源码？

内部技术决策稿 · 深度修订版 V3

研究基线：OpenCode dev commit cc4b4561（同步 v1.18.18，2026-08-13）

01 先给结论：能不能？

必须先区分 “DeepSeek V4 Flash 0731 + 两套 Harness” 与 “Claude Code + 原生 Claude” 两种完全不同的目标。

结论 A - 同一个 iAgent/standard (DeepSeek V4 Flash 0731)：可以把“统计持平”作为现实工程目标

在同一模型服务实例、同一任务、同一预算、同一环境，并把 Claude Code 的协议代理层纳入观测的前提下，OpenCode 通过 Harness 定向优化达到 “公司目标 workload 上统计等效/持平” 有现实依据。外部同一本地 DeepSeek V4 Flash 实验在 focused bug-fix 上没有测出显著质量差距，主要差异落在探索、委派、Token 和耗时；但该外部模型不是公司 0731，因此只能支持方法论，不能替代内部结果。[E1]

结论 B - 全面超过 Claude Code：不能承诺

“全面超过” 没有稳定定义，而且 Claude Code 是持续演进的闭源 Harness；如果比较对象升级为 Claude Code + 最新原生 Claude/Opus，则模型上限、原生协议、模型-Harness 联合优化都会移动目标。Harness 可以放大冻结模型，但不能无限突破 backbone capability bound。[P4]

真正建议

不要先 Fork 大量 OpenCode 核心代码。先构建 same-model trajectory benchmark，把差异定位到 Prompt/Tool/Delegation/Context/Verify/Stop 六个机制，再以 feature flag 做单变量 ablation。首批改造应集中在 “iAgent Model Profile + 任务感知工作流 + 验证/停止控制”，而不是先做多 Agent。

目标	判断	为什么
DeepSeek V4 Flash 0731 + OpenCode ≈ DeepSeek V4 Flash 0731 + Claude Code（公司 workload）	可行，值得做	模型变量基本固定，Harness 是主要可控变量；已有同类 same-model 证据。
所有公开 benchmark 都赢 Claude Code	不应设为目标	benchmark 覆盖有限、版本变化快、基础设施噪声可达多个百分点。
DeepSeek V4 Flash 0731 + OpenCode ≥ Claude Code + 原生 Opus	不能承诺	此时既换 Harness 又换模型，模型能力上限与联合优化不可控。

02 先把 iAgent/standard 说清楚

关键纠正：iAgent/standard 不是泛指某个“Flash 模型”，而是公司内部路由别名；本报告固定把它定义为公司本地部署的 DeepSeek V4 Flash 0731。

项目	本报告定义	必须内部测量/确认
模型别名	iAgent/standard（公司路由别名）	仅内部约定；外部不可验证
底层模型	DeepSeek V4 Flash 0731	锁定具体权重/构建版本、推理参数、量化与服务镜像；不要只记录别名
部署	公司本地部署	服务版本、量化、GPU、KV cache、并发策略
上下文	未知，必须实测	max context / usable context / overflow 行为
工具调用	未知，必须实测	OpenAI/Anthropic tool-call 兼容度、parallel call、schema 限制
推理参数	未知，必须锁定	temperature/top_p/effort/reasoning 开关/output limit

“同一个模型”比想象中更严格

不能只保证两个 CLI 都显示 iagent/standard。必须保证二者最终打到同一模型服务版本，并记录 Harness/Proxy 处理后的实际 system messages、tool schema、sampling 参数、context 截断、stop reason 与返回格式。否则比较的可能是 Adapter，不是 Harness。

建议的比较单位

$$\text{Result} = f(\text{Model}, \text{Harness}, \text{Adapter}, \text{Environment}, \text{Budget})$$

- Model：同一个 DeepSeek V4 Flash 0731 服务实例、权重/构建版本与推理配置。
- Harness：OpenCode vs Claude Code 的 Prompt、工具、循环、上下文与子 Agent。
- Adapter：OpenCode Provider 路径；Claude Code 若通过 Anthropic→内部 API 代理，则代理本身必须单独观测。
- Environment：相同 repo snapshot、CPU/内存/容器、依赖、网络、权限。
- Budget：相同 wall-clock、turn/tool-call/token 上限，或明确报告预算差异。

03 调查方法：从“功能列表”改为“ workflows与因果”

证据按可靠性分层；任何无法从公开资料确认的 Claude Code 内部细节都标记为未知。

证据层	来源	本报告怎么用
A - 一手源码	OpenCode dev 源码 (v1.18.18)	重建真实 Prompt→Loop→Tool→Processor→Compaction 执行链；定位具体代码改造点。
A - 官方公开	Claude Code 官方 How it works / Context / Hooks / MCP / Subagents	只陈述官方公开的行为与能力，不把第三方反编译当“内部真相”。
B - 受控 same-model	Nawk: 本地 DeepSeek V4 Flash × OpenCode/Claude Code/Pi	回答“同模型换 Harness 会发生什么”；最接近本项目。
B - Harness 研究	The Scaffold Effect / Prompt-Induced Waste / Infrastructure Noise	回答 Harness 对 token、latency、failure fingerprint 与评测噪声的影响。
C - 综合 benchmark	Artificial Analysis	提供同模型跨 Harness 的外部横截面，但不作为内部结论。
D - 博客/用户反馈	实践博客、论坛	只用于提出假设，不用于宣布因果。

方法论变化

问题不再是“Claude Code 有什么、OpenCode 少什么”，而是：同一任务 T、同一模型 M，在 Harness H₁/H₂ 下分别生成怎样的 trajectory τ ？哪个机制改变了 τ ，最终如何改变 verified success、time、tokens、hang 与人工介入？

本次实际读取的 OpenCode 核心代码

源码文件	在工作流中的角色
session/system.ts	按 model.api.id 选择模型 Prompt；注入 env / references / skills / MCP。
session/llm/request.ts	组装 system、messages、tools、采样参数与 headers。
session/prompt.ts	核心 while-loop：pending task / compaction / reminders / tools / LLM / continue。
session/processor.ts	处理 stream、工具调用、retry、snapshot、doom-loop、finish。
session/compaction.ts	overflow、tail 保留、summary、旧 tool output pruning。
agent/agent.ts + agent/prompt/explore.txt	build/plan/general/explore/summary/compaction 等 Agent 定义。
tool/task.ts	子 Agent 子会话创建、foreground/background、返回摘要。
tool/registry.ts + session/tools.ts	工具集合、模型 ID 分支、schema/permission/MCP 暴露。

04 OpenCode 的真实工作流程

不是推测：下面由当前 dev 分支核心源码重建。

OpenCode 实际执行链（dev / v1.18.18）

基于 session/{prompt,processor,llm,compaction}.ts、tool/registry.ts 与 task.ts



图 1 OpenCode 核心执行链（源码级抽象）。[OC2][OC3][OC4][OC5]

关键控制点

`prompt()` 保存用户消息后进入 `runLoop()`；循环先读取已 compaction 的消息，再处理 pending subtask / compaction。[OC3]

每轮重新解析 Agent、工具、权限、skills/MCP/instructions，并把它们组装进下一次模型请求。[OC2][OC3][OC8]

`SessionProcessor` 记录 reasoning/text/tool stream、snapshot 与 patch；同一工具+同一输入连续出现 3 次才触发 doom-loop 询问。[OC4]

正常停止主要依据模型的 finish reason 与是否仍有 tool calls；当前核心 Loop 没有一个通用的“任务验收通过才允许 Stop”的硬 Gate。[OC3][OC4]

上下文溢出时进入 compaction；结束后还会异步 prune 旧 tool outputs。[OC5]

第一条源码级判断

OpenCode 已经是成熟 Harness：有 Agent、subtask、plan、retry、snapshot、compaction、MCP、skills 和插件。追平 Claude Code 不应从“重写 Agent Loop”开始，而应先让这些机制显式适配 DeepSeek V4 Flash 0731，而不是继续依赖通用 prompt 与 model-id 字符串启发式。

05 源码发现 ①：DeepSeek V4 Flash 0731 在 OpenCode 中目前不是 “一等模型族”

这是新版最值得先做 A/B 的低成本、高潜在收益假设。

Finding 1 - DeepSeek V4 Flash 0731 当前没有专用 System Prompt 分支

`session/system.ts` 的 `SystemPrompt.provider()` 只对 muse / gpt / gemini / claude / trinity / kimi 等模型族选择专用 prompt，源码里没有 DeepSeek 分支。因此：即使 `model.api.id` 暴露为 `deepseek-v4-flash-0731`，也会落到 `PROMPT\_DEFAULT`；如果公司侧只暴露 `standard`，同样落 default。除非通过 `agent.prompt` 或插件显式覆盖。这是源码事实，不只是“内部别名可能选错”的猜测。[OC1]

Finding 2 - 工具形态也通过模型 ID 猜

`tool/registry.ts` 当前用 modelID 是否包含特定 `gpt-` 字样决定暴露 `apply\_patch` 还是 `edit/write`。因此模型别名不只是影响 Prompt，还可能影响编辑工具形态。[OC8]

Finding 3 - 自定义 Agent Prompt 会覆盖 provider prompt

`session/llm/request.ts` 的组装逻辑是：有 agent.prompt 时优先使用它，否则才调用 SystemPrompt.provider(model)。这意味着某些自定义 Agent 可能完全绕过模型家族 Prompt。[OC2]

建议改法：从字符串分支升级为显式 ModelProfile

字段	示例（待公司实测确定）	目的
profile_id	iagent-flash-0731	稳定标识，不依赖对外别名。
prompt_profile	flash0731-v1	选择经过 A/B 的 system prompt。
edit_tool	edit_write / apply_patch	按模型真实工具调用表现选择。
parallel_tools	true/false	决定是否鼓励批量并行工具。
tool_schema_budget	small / medium	控制可见工具数量与 schema 大小。
subagent_policy	direct-first / adaptive	避免 focused task 无收益委派。
context_profile	threshold + preserve rules	针对本地模型的长上下文与 KV cache 特性。

P0 实验

不需要先改大 Core。可先用 custom agent prompt + plugin `experimental.chat.system.transform` + permissions/tool definitions 做原型；只有在 same-model A/B 中确认收益，才把 ModelProfile 下沉到 core。

06 源码发现 ②：OpenCode 的默认 Prompt 明确“偏好 Task 做文件搜索”

这会直接塑造 DeepSeek V4 Flash 0731 的工作流，而不是一个中性的实现细节。

默认 Prompt 的策略倾向

`session/prompt/default.txt` 明确建议：做文件搜索时优先使用 Task tool 以减少主上下文；同时鼓励独立工具并行。[OC9] 这会把“搜代码”从主 Agent 的连续推理，转换为 child session 的委派与摘要回传。

Task tool 的真实代价

`tool/task.ts` 会创建/恢复一个 parentID 指向主会话的 child session，并让 child 调用完整 `ops.prompt()`，即重新跑一套 OpenCode Loop。[OC7]

默认 foreground 子任务需要主 Agent 等待结果；background 仍是实验能力开关。[OC7]

子 Agent 返回的是任务结果/摘要，不会把完整子会话逐 token 注入父会话；这能降低父上下文，但会增加 handoff latency 和信息压缩风险。

`explore` Agent 本身是一个搜索专家，允许 glob/grep/read/bash/web 搜索，禁止写文件。[OC6]

为什么对 DeepSeek V4 Flash 0731 特别重要

如果 focused bug-fix 本来只需 2-3 次 grep/read 就能定位，先 spawn Explore/General 可能只是协调税；如果任务很宽、搜索结果会污染主上下文，隔离子 Agent 才产生收益。因此应该“任务感知委派”，而不是全局偏好委派。

任务类型	建议策略	原因
Focused bug / 已知模块	Direct-first: 主 Agent grep/read/edit/test	减少 child session + handoff; 保留连续局部推理。
Ambiguous localization	允许 1 个 Explore, 设 search budget	把广搜索隔离, 但限制无止境探索。
跨模块 / research + code + test	可并行多个独立子任务	任务宽度足以摊薄 coordination cost。

07 Claude Code 的公开工作流程：Gather → Act → Verify

这里严格使用官方公开资料；闭源细节不做臆测。

Claude Code 公开 workflow：Agent Loop

依据官方 How Claude Code Works / Context / Hooks / Subagents 公开资料

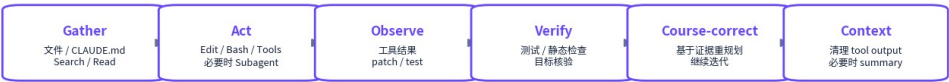


图 2 Claude Code 官方对 agentic loop 的公开描述。[CC1]

官方把核心循环明确表述为 gather context → take action → verify results，并根据上一步证据不断 course-correct。
[CC1]

上下文包括 conversation、文件、command output、CLAUDE.md、auto memory、skills、system；接近上限时先清旧 tool outputs，再摘要。[CC1][CC2]

Subagent 有独立 context，适合把会淹没主会话的大量 search/log/file 读取隔离出去；完成后只回传 summary。[CC1]
[CC5]

MCP Tool Search 默认延迟工具 schema，仅在实际需要时加载；但当 `ANTHROPIC\_BASE\_URL` 指向非一方代理时，官方文档说明默认会关闭这种 deferral，除非显式覆盖。[CC4]

Hooks 可在 Pre/PostToolUse、Stop、SubagentStop、PreCompact 等节点执行确定性脚本、prompt verifier 或 agent verifier；Stop hook 甚至可以阻止 Agent 停止并把“未完成原因”喂回下一轮。[CC3]

重要边界

Claude Code 的官方产品目标是 Claude 模型。把它通过公司代理接到 DeepSeek V4 Flash 0731 时，不能假设所有 Claude-native 特性仍保持同样行为；例如 tool_reference / deferred MCP schema 就受代理兼容性影响。因此公司评测必须捕获“代理之后真正送给 DeepSeek V4 Flash 0731 的 payload”。

08 两边工作流到底差在哪里？

不是谁“功能多”，而是哪种默认策略会改变 trajectory。

机制	OpenCode 当前源码	Claude Code 公开行为	对 DeepSeek V4 Flash 0731 的潜在影响
模型适配	SystemPrompt.provider() 没有 DeepSeek 分支， DeepSeek 会落 PROMPT_DEFAULT; ToolRegistry 仅对特定 gpt-* 切 apply_patch， DeepSeek 默认仍走 edit/write。	围绕 Claude 模型设计；公开文档 不暴露完整内部 prompt	公司 0731 当前不是 OpenCode 的“一等模型 族”。Prompt、tool primitive、schema 与参 数应显式 Profile 化，而不 是继续依赖 model-id 字符 串启发式。
探索	默认 prompt 鼓励文件搜索优先 Task；Explore 是独立 subagent	核心 loop 先 gather context； subagent 用于隔离会淹没主 context 的工作	OC 可能过早委派；CC 可能过度 探索。哪个更好取决于任务宽度。
工具暴露	内置工具 + MCP；当前每轮 resolve 并构造 schema	MCP tool schema 可默认 deferred/on-demand	DeepSeek V4 Flash 0731 对 tool 数量/描 述/schema 的敏感度需要 内部 A/B；若敏感，工具 面会直接影响选择准确率 与 prefix 成本。
验证	prompt 要求 test/lint；core stop 仍主要由 finish/tool-call 驱动	公开 loop 把 Verify 作为一等阶 段；Stop hooks 可外部阻止停止	弱/快模型容易“觉得完成”；硬 verifier 可能是高杠杆差异。
停机/循环	同 tool+同 input 连续 3 次触发 doom-loop；无通用 progress score	Stop/StopFailure + hooks；可通 过 verifier 阻止停止	OC 对“换着工具空转”检测不 足；也可能过早 stop。
上下文	tail + summary + old tool-output prune；摘要策略通用	先清旧 tool output，再 summary；CLAUDE.md/memor y/skills 重新注入	语义保真、state preservation、 tool schema 占用会改变后续行 为。
子 Agent	child session 调同一完整 OC loop；default depth 1	独立 context；可并行/teams；只 回 summary	同一 Flash 多 Agent 不提升模型上 限，只在隔离/并行能回本时有价 值。

最核心差异

OpenCode 更“开放且通用”：它必须适配很多模型，所以策略不可避免偏通用；Claude Code 更“模型/产品协同”：公开工作流把 context→action→verify 打磨成默认体验。对公司自有 DeepSeek V4 Flash 0731 来说，机会恰恰是把 OpenCode 从“通用 Harness”变成“针对这一模型和 workload 的专用 Harness”。

09 最接近本项目的外部实证：同一本地 DeepSeek V4 Flash 换 Harness（非公司 0731）

这比 Claude/Opus vs OpenCode/其他模型的榜单更有解释力。

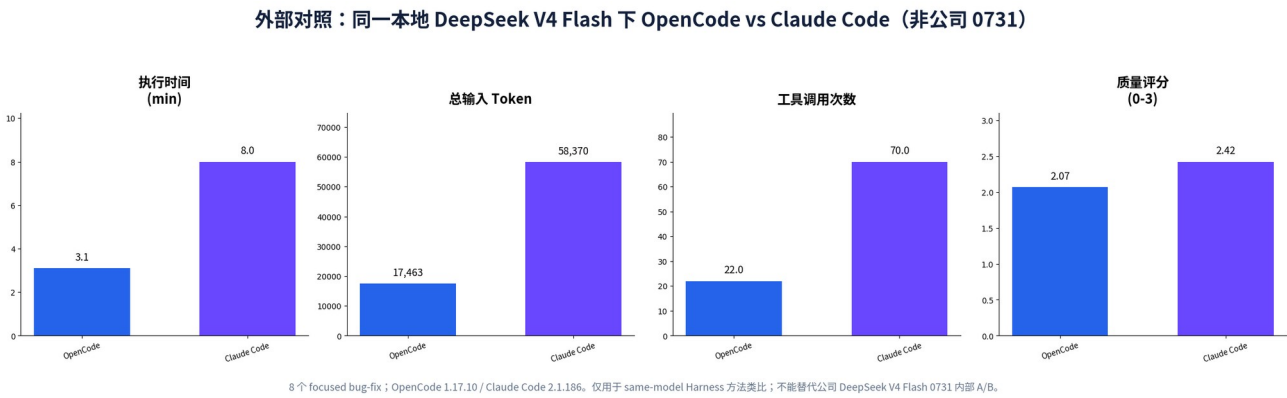


图 3 Nawk 同一本地 DeepSeek V4 Flash 实验的 OpenCode vs Claude Code 指标。注意：质量均值不同，但置信区间完全重叠。[E1]

观察	OpenCode	Claude Code	对本项目的含义
平均质量(0-3)	2.07	2.42	点估计不能当排名；作者报告 95% CI 全部重叠。
平均耗时	3.1 min	8.0 min	同一 Flash 下，Claude 的更重探索未换来显著质量收益。
平均输出 tokens	17,463（含 subagent）	58,370	workflow 决定生成/工具开销。
约工具调用	~22	~70	Claude 搜索/读取更重，首个编辑更晚。
行为特征	delegates / handoff	explores deeply / edits late	正好与 OC default Task preference 的源码吻合。

不能过度解读

该实验使用 DeepSeek V4 Flash，但公开材料没有证明它与公司的 DeepSeek V4 Flash 0731 是同一 build / 权重 / 推理服务；OpenCode 版本为 1.17.10，而本报告读取的是 1.18.18，且只有 8 个 focused bugs。它只证明“固定同一 DeepSeek V4 Flash 后，Harness 可以主要改变效率与轨迹”，不能直接证明公司 0731 上也会得到同样结果。

10 为什么博客会互相矛盾？这正是必须内部 A/B 的原因

同样是“同模型比较”，不同版本、代理、任务、计数口径会把结论反转。

证据	设置	结果	我们应该学什么
Nawk 2026-07 [E1]	本地 DeepSeek V4 Flash；8 bug-fix；多次重复	质量无显著差异；OC 比 CC 更快、更少输出 token	focused bug 下 heavy exploration 未必有收益。
Eishan 2026-06 [E2]	DeepSeek V4 Flash / Nemotron；12 小 Python task；OpenRouter	Claude Code ~52-55k token/solved；OpenCode ~72-80k	另一个 Provider/任务/版本下，OC 反而更重。
Scaffold Effect [P1]	Qwen3.6/MiniMax × Goose/OC/OpenHands；Terminal-Bench Pro	pass 差距相对小，但 token/solved 可差 40×；OC 有 idle-loop/TIME fingerprint	Harness 对效率和失败模式的影响很稳定，但方向依赖实现。
Artificial Analysis [B1]	同 Opus 4.7 medium	OC Index 50 vs CC 40，但 OC 更慢、token 更多	“更高分”不等于“工程上更优”，需要 Pareto 指标。
Anthropic Infra Noise [P3]	Terminal-Bench 2.0 环境资源变化	基础设施可造成约 6pp 摆动	小差距不能靠单次跑数宣布胜负。

因此新版结论

外部资料的作用不是替你们给出“OpenCode/Claude Code 谁强”的答案，而是告诉我们应该观测哪些变量、哪些 failure mode、哪些改动值得优先试。最终结论只能来自公司 DeepSeek V4 Flash 0731 + 公司 repo + 公司协议代理的 paired benchmark。外部 DeepSeek V4 Flash 数据一律标为“方法类比证据”，不能混称为 0731 实验。

11 是什么导致了结果差异？完整因果链

同一个模型不代表同一个条件分布：Harness 每一轮都在改变下一轮输入。

为什么同一个模型会产生不同结果？

Harness 每一轮都改变下一轮输入 - 差异会沿 trajectory 累积并放大



$$\text{Result} = f(\text{Model} \times \text{Harness} \times \text{Adapter} \times \text{Environment} \times \text{Budget})$$

因此“同一个 DeepSeek V4 Flash 0731”只是固定了 Model，不能自动消除 Harness 差异。

图 4 Harness 通过输入、策略、状态和控制层改变 trajectory，差异随多轮迭代放大。

六个最可能的“结果差异源”

- 1. Model Profile 错配：** 别名导致 generic prompt / generic tool choice；这是当前源码可直接验证的首要假设。
- 2. Delegation 策略：** OpenCode default prompt 偏向 Task 搜索；focused task 会产生 child-session handoff 税。
- 3. Tool Surface：** 每个 tool schema 都会占 prefix/context，并影响 action distribution。DeepSeek V4 Flash 0731 对 tool 数量、描述长度与 schema 复杂度是否更敏感，必须通过内部 A/B 实测，而不能凭模型“Flash”标签推断。
- 4. Explore → Edit 时机：** 搜索过少会定位错；搜索过多会耗时/空转。Nawk 观察到 CC 约 80% 进程后才首次编辑。
- 5. Verify / Stop：** 没有 acceptance gate 时，模型可能提前宣布完成；没有 progress detector 时，又可能换着工具继续空转。
- 6. Context / Adapter：** 摘要、prune、proxy 转换、tool-call dialect 与缓存共同决定模型真正看到的 state。

12 为什么“能”持平：五条硬理由

这里的“能”只指同一个 DeepSeek V4 Flash 0731、指定公司 workload 的统计等效。

模型变量被锁定

如果两边最终都是同一个 DeepSeek V4 Flash 0731，模型参数能力不再是主要解释变量；剩下的差异更多来自 Harness/Adapter/Environment。

OpenCode 已有完整可改的 Harness 基础

源码已具备 Loop、Task/Subagent、Plan、Retry、Snapshot、Compaction、Plugin、Permission、MCP；不需要从零造 Claude Code。[OC3][OC4][OC5][OC7]

已有 same-local-Flash 证据

在 focused bug-fix 上，Nawk 未测出 OpenCode/Claude Code 的显著质量差异；说明“持平”并非纯理论愿望。[E1]

Harness 的确是强杠杆

Scaffold Effect、Prompt-Induced Waste 等研究证明同一模型换 Harness 会大幅改变 token、latency、failure mode；最新 HSI 还显示冻结 DeepSeek-V4-Flash-Preview 可以通过 task-specific harness evolution 在中等难度任务上显著提升。[P1][P2][P4]

OpenCode 对内部模型反而有定制优势

Claude Code 必须围绕 Claude 的产品兼容性演进；OpenCode 可为公司 DeepSeek V4 Flash 0731 建专用 ModelProfile、toolset、stop/verify 策略和 context ledger。

13 为什么“不能”直接承诺：五个边界

批判性结论：Harness 能放大模型，但不是魔法。

Backbone capability bound

如果 DeepSeek V4 Flash 0731 在某类 hard SWE task 上本身无法形成正确假设，增加更多 Loop/Agent 可能只是在更昂贵地绕圈。Eishan 的 SWE-bench Lite 实验显示同一模型在多个 Harness 上只解出同一题；HSI 也明确观察到超出 backbone 能力时 harness evolution 无法提升。[E2][P4]

Claude Code 是移动目标

官方持续新增 context、hooks、agents、teams、tool search 等机制；你今天追平的是一个版本，不是永久能力。

Claude Code + 本地 Flash 不是官方原生路径

代理层可能禁用/改变 tool_reference、MCP deferral、message/tool dialect。若代理不透明，不能把差异都归因给 Harness。[CC4]

OpenCode 改动存在非线性连锁

Prompt 改变 tool choice → tool output 改变 context → compaction 时机改变 → next action 改变。必须单变量 ablation，不能一次性“重构整个 Agent”。

没有万能 benchmark

不同 workload 会得出相反效率结论；基础设施本身可产生约 6pp 变化。只能对“公司 workload”做等效性声明。[P3]

14 建议改造架构：薄适配层优先，不做“大 Fork”

核心目标是给 DeepSeek V4 Flash 0731 一套可测、可回滚、可逐步下沉的 Harness Profile。

推荐：不“重写 OpenCode”，而是增加 DeepSeek V4 Flash 0731 专用 Harness Profile

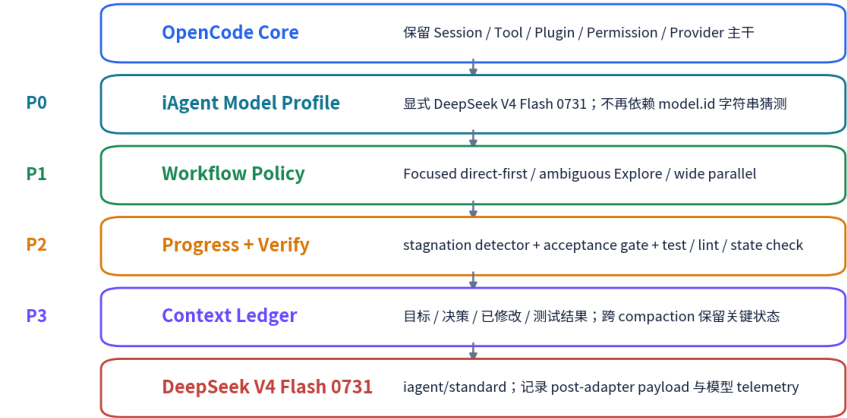


图 5 推荐的 iAgent-specific Harness 分层。

优先级	改造机制	先在哪验证	成功后下沉到
P0	ModelProfile: prompt/tool/schema/context 能力显式声明	custom agent + system transform + config	session/system.ts, llm/request.ts, tool/registry.ts
P1	Task-aware workflow: direct / explore / wide	专用 agent prompt + task permissions	prompt/default 或新 iagent-flash prompt; task.ts
P2	Progress + Verify Gate	plugin tool hooks + external acceptance script	processor.ts / reminders.ts / stop controller
P3	Context Ledger + compaction preserve	plugin compact context + structured state file	compaction.ts / summary pipeline
P4	Selective multi-agent	仅 broad task A/B	task.ts / background jobs / routing

15 具体改哪几处 OpenCode：源码级改造清单

把“为什么改”与“回归风险”写在一起，避免盲改。

文件	建议	直接原因	连锁风险 / 控制
session/system.ts	新增 capability/profile 映射，去掉只靠 api.id 猜模型	iagent alias 落 generic prompt	所有 provider prompt 选择；必须 feature flag
session/llm/request.ts	记录最终 system/tool/params hash；支持 profile overlay	看不到模型实际收到什么	请求前缀、cache、参数；先观测后修改
tool/registry.ts	按 profile 选择 edit/patch 与最小工具集	当前工具分支对 internal alias 无感	工具兼容性；逐工具 A/B
session/tools.ts	tool budget / lazy exposure / schema metrics	Flash 可能被过多 tool schema 干扰	MCP/插件兼容；保留 fallback
prompt/default.txt / 新 prompt	把“搜索优先 Task”改为 task-aware direct-first	focused bug 委派产生 handoff tax	复杂任务搜索隔离；需要 router
tool/task.ts	给 delegation 增加宽度/独立性判定与预算	同模型子 Agent 未必增加质量	并行能力；不要全局禁用
session/processor.ts	加入 semantic progress / no-progress turns / finish verifier seam	当前 doom-loop 只识别同工具同输入 ×3	误杀有价值循环；需软阈值+回退
session/reminders.ts	注入 acceptance criteria / verify reminder（只在必要时）	模型可能过早 finish	额外 token；只在 edit 后或 stop 前
session/compaction.ts	独立保留 task ledger：目标/改动/错误/测试/决策	通用 summary 可能丢任务状态	context 变大；ledger 必须短且结构化

原则

任何 P0-P3 改造都必须可以通过 flag 关闭；不要把多个改动合并成一个 PR 再看总分，否则无法知道因果贡献。

16 公司应该怎么评：OpenCode vs Claude Code

Harness Benchmark v1

目标不是造一个漂亮排行榜，而是回答“哪个机制让 DeepSeek V4 Flash 0731 成功/失败”。

Cell	Harness	Model/Adapter	用途
A	Claude Code	DeepSeek V4 Flash 0731 + 公司 Claude→模型协议代理	reference baseline；完整记录代理后 payload
B	OpenCode baseline	DeepSeek V4 Flash 0731 (直连/同网关)	当前能力基线
C	OpenCode modified	DeepSeek V4 Flash 0731；一次只开一个 feature flag	因果 ablation
D（诊断）	极简 Harness / 自研 4-6 tool loop	DeepSeek V4 Flash 0731	识别模型 ceiling 与 Harness overhead；不是产品方案

任务集：优先从公司历史 GitLab 构建 Private Benchmark

类别	为什么需要	核心 verifier
Focused bug-fix	验证 direct-first / delegation 税	hidden tests + diff scope
Repo localization / Q&A	单测探索与 context selection	gold file/line + answer
Multi-file feature	测试“宽任务”与并行价值	acceptance tests + API contract
Refactor / migration	测试跨文件一致性与 long horizon	full regression + type/lint
Test evolution	防止只修代码不更新 missing/stale tests	hidden test-intent verifier
CI / config / infra	覆盖命令、环境、权限与恢复	containerized end-to-end check

指标：必须同时报告质量、效率、可靠性

质量	效率	可靠性 / 行为
verified pass@1 / pass@n	wall time / P95	timeout / hang / retry
hidden tests / spec satisfaction	input+output tokens (含 child sessions)	human intervention
file/line localization recall	tool calls / time-to-first-edit	invalid tool / repeated action
regression scope / maintainability	fixed prefix / schema tokens	verification coverage / stop reason

17 如何定义“持平”：用等效性，不用肉眼看均值

只要差距落在噪声里，就不能宣布 Claude Code 更强或 OpenCode 更强。

建议的统计口径

每个任务 paired 运行多个随机种子；报告 bootstrap 95% CI。业务先定义可接受等效区间 δ （例如 pass rate $\pm 3\sim 5pp$ 只是示例，不是通用标准），再做 equivalence / non-inferiority 判断。Anthropic 已证明基础设施配置可让 coding eval 摆动约 6pp，因此小于这一量级的单次差异必须极其谨慎。[P3]

判定	条件（示意）	动作
质量持平	$\Delta Pass$ 的 95% CI 完全落入 $\pm \delta$	再比较 time/token/reliability，选择 Pareto 更优者。
质量不劣 + 效率更优	质量满足 non-inferiority；time/token 显著下降	优先合并该 Harness 改造。
质量更高但代价暴涨	Pass $\uparrow$ ，但 token/time/hang 同时显著 $\uparrow$	不直接宣布“超过”；看公司成本与 SLA。
两边同样失败	相同 hard tasks 在 minimal/CC/OC 都失败	高度怀疑 backbone ceiling；停止堆 Harness。
结果反复翻转	seed/环境变化导致排名变化	增加重复数、固定 infra、检查代理/缓存。

“超过 Claude Code” 不是一个分数，而是一条 Pareto 前沿：Quality × Time × Token × Reliability × Human Oversight

18 必须补的基础设施：Agent Trajectory Profiler

不先把轨迹记录下来，就永远只能猜 Claude Code 为什么成功、OpenCode 为什么失败。

层	每轮记录什么	用来回答什么
Request	system hash/size、tool schema size/list、params、adapter、cache header	是不是 prompt/tool/proxy 一开始就不同？
Explore	search/read 文件、命中 gold file/line、重复读取	是谁更快找到正确上下文？
Delegation	subagent spawn、child token/time、handoff summary size	Task 委派到底省 context 还是只增 latency？
Action	tool call、args、失败/repair、首次 edit 时机	动作策略从哪一轮分叉？
Context	每轮 token、prune/compact 边界、保留 ledger	错误是否发生在 context 丢失之后？
Verify	tests/lint/typecheck、失败→修复次数、stop 前 evidence	是否真的验证，还是“觉得完成”？
Outcome	pass、time、token、hang、人工介入	最终工程价值。

OpenCode 侧实现成本不高

当前 `llm.ts` 已有 OpenTelemetry seam，plugin 也有 system/tool/chat hooks；可以先做 recorder 而不改核心 Loop。Claude Code 侧可以结合本地 JSONL transcript、Hooks 与网关 HTTP recorder。最重要的是在公司代理之后再记一份 post-adapter payload。[OC10][CC3]

强制计数口径

OpenCode 必须把 child subagent 的 token/time 重新加回 parent task，总账不能只看主会话；Nawk 的实验正是因为补回子 Agent token 后，OpenCode 才从“约 11k”变为约 17k。[E1]

19 怎么控制“改一处引发连锁反应”？

把 OpenCode 当非线性闭环系统，而不是普通 CRUD 项目。

风险	典型链路	控制手段
Prompt 改动	prompt → tool choice → context → compaction → stop	每个 prompt 版本有 hash；固定模型/任务；单变量 A/B。
工具裁剪	少一个 tool → 模型改走 Bash/错误替代路径	tool deny 先做软实验；记录 fallback/invalid call。
Verifier 过强	stop 被阻止 → loop 变长 → token/hang 上升	只在发生 edit 后启用；限制 verifier 次数；可降级。
Progress detector 误杀	不同工具但有真实探索进展被判 stagnation	用 evidence novelty 而非纯 turn count；soft replan 优先于 terminate。
Compaction 改动	摘要更短 → 丢决策；更长 → context 压力	结构化 ledger + 原 summary 并存做 A/B。
Subagent routing	更多并行 → 重复工作/冲突/本地服务拥塞	只有独立 broad subtasks 才并行；限制 concurrency。

Feature Flag 设计

```
experimental:
  iagent_model_profile: flash0731_v1
  workflow_policy: adaptive_v1
  direct_search_first: true
  verify_gate: shadow
  progress_detector: shadow
  context_ledger: false
```

Shadow mode 很重要

Verifier / progress detector 第一阶段只打日志、不改变行为。先看“如果它有权阻止，多少次会阻止？阻止是否真的对应失败？”再切到 active。

20 推荐推进顺序：先诊断，再专用化，再决定是否动 Core

每一步都有明确“继续/停止”条件。

阶段	做什么	继续条件	停止/回退条件
0. Baseline	锁定 DeepSeek V4 Flash 0731；录 CC/OC post-adapter payload；建 private tasks	可重现、轨迹字段齐全、噪声已量化	代理/环境仍不一致时禁止比较。
1. Model Profile	显式 prompt/tool/schema profile；修复 internal alias 字符串猜测	同任务质量/工具正确率/效率出现稳定 Pareto 改善	无收益则不下沉 core。
2. Workflow Policy	focused direct-first；ambiguous single explore；wide parallel	减少 handoff/time，且不降低 localization/pass	复杂任务回归则仅对 focused tasks 启用。
3. Verify + Progress	shadow→active acceptance gate；stagnation replan	过早 stop / idle-loop 下降，质量不劣	token/time 大涨或误阻止率高。
4. Context Ledger	结构化保存 objective/decision/modified/ tests/errors	长任务 compaction 后成功率/恢复显著改善	短任务没有收益则按阈值启用。
5. Selective Multi-agent	只对 broad independent work 并行	总 wall-clock 下降且冲突/重复低	同模型 ceiling 明显、协调税超过收益。

最重要的停止规则

如果某一类 hard task 在 Claude Code + DeepSeek V4 Flash 0731、OpenCode + DeepSeek V4 Flash 0731、以及极简 Harness + DeepSeek V4 Flash 0731 上都稳定失败，而且 failure analysis 指向同一个 reasoning gap，就应把它归为“模型上限候选”，不要继续往 OpenCode 里堆 Prompt/Agent。

21 最终建议：公司例会应该怎么回答“能不能”

把技术结论转成可执行的决策语言。

一句话答案

能：如果目标是“同一个公司本地 DeepSeek V4 Flash 0731，在公司定义的 Coding workload 上，让 OpenCode 的 verified quality 与 Claude Code 统计持平”，这是可行且值得验证的。不能：如果目标是“通过改 OpenCode 保证在所有任务上全面超过 Claude Code + 最新原生 Claude”，这个承诺没有科学依据。

为什么值得做

OpenCode 当前差异已能落到具体代码机制，而不是模糊的“Claude magic”：model profile、Task delegation、tool surface、verify/stop、compaction。

外部同一本地 DeepSeek V4 Flash 证据显示，focused bug-fix 上 OpenCode 与 Claude Code 可能已经接近质量等效；但外部实验不是公司 0731，只能说明“不必先大重构，也可能通过 Harness 调整看到价值”。[E1]

针对内部模型做 Harness 专用化，是 OpenCode 开源定位相对 Claude Code 的真正优势。

为什么必须谨慎

Claude Code 通过公司代理接本地模型时，Adapter 本身就是变量；不抓 post-adapter payload，任何“能力差距分析”都可能错因。

不同 same-model 实验对 OpenCode/Claude Code 的效率结论会反转，说明版本/工作负载/工具/代理比博客口碑更重要。[E1][E2]

模型上限不可由 Harness 无限突破；应设置明确停止条件。[P4]

建议立即立项的三个技术课题

课题	核心问题	第一验收
A. iAgent ModelProfile	`standard` 别名是否导致 prompt/tool 适配错配？	显式 profile vs baseline same-model A/B。
B. Direct-first Workflow	focused bug 搜索是否不应默认 Task delegation？	time-to-first-correct-edit ↓，pass 不劣。
C. Verify/Stop Controller	过早 stop / 空转是否是主要 failure mode？	shadow verifier 命中与真实失败高度相关，再 active。

最终技术方向：不要复制 Claude Code；让 OpenCode 成为“最懂公司 DeepSeek V4 Flash 0731 的 Harness”。

22 主要来源与可追溯性

OpenCode 源码按 2026-08-13 dev commit cc4b4561; Claude Code 只引用公开官方资料。

- [OC1] OpenCode source: packages/opencode/src/session/system.ts - [source](#)
- [OC2] OpenCode source: packages/opencode/src/session/llm/request.ts - [source](#)
- [OC3] OpenCode source: packages/opencode/src/session/prompt.ts - [source](#)
- [OC4] OpenCode source: packages/opencode/src/session/processor.ts - [source](#)
- [OC5] OpenCode source: packages/opencode/src/session/compaction.ts - [source](#)
- [OC6] OpenCode source: packages/opencode/src/agent/agent.ts + agent/prompt/explore.txt - [source](#)
- [OC7] OpenCode source: packages/opencode/src/tool/task.ts - [source](#)
- [OC8] OpenCode source: packages/opencode/src/tool/registry.ts + session/tools.ts - [source](#)
- [OC9] OpenCode source: packages/opencode/src/session/prompt/default.txt - [source](#)
- [OC10] OpenCode dev commit cc4b4561, sync release versions for v1.18.18 - [source](#)
- [CC1] Claude Code Docs - How Claude Code works - [source](#)
- [CC2] Claude Code Docs - Explore the context window - [source](#)
- [CC3] Claude Code Docs - Hooks reference / guide - [source](#)
- [CC4] Claude Code Docs - MCP / Tool Search - [source](#)
- [CC5] Claude Code Docs - Subagents / parallel agents - [source](#)
- [E1] Nawk (2026-07) - Your agent harness is an efficiency decision, not a quality decision - [source](#)
- [E2] Eishan Lawrence (2026-06) - I tested 12 coding harnesses for efficiency - [source](#)
- [P1] Vats & Golev (2026) - The Scaffold Effect in Coding Agents, arXiv:2607.22585 - [source](#)
- [P2] Weinberger & Hozes (2026) - Prompt-Induced Waste..., arXiv:2608.01347 - [source](#)
- [P3] Anthropic (2026-02) - Quantifying infrastructure noise in agentic coding evals - [source](#)
- [P4] Zhou (2026-08) - Hierarchical Self-Improvement..., arXiv:2608.08466 - [source](#)
- [B1] Artificial Analysis - Claude Code vs OpenCode Coding Agent Comparison - [source](#)
- [S1] Code as Agent Harness survey, arXiv:2605.18747 - [source](#)

阅读提示

外部 DeepSeek V4 Flash 证据只用于 same-model Harness A/B 的方法类比；它不是公司 DeepSeek V4 Flash 0731 的实验结果。公司最终结论必须由内部 0731 服务、公司 repo、公司协议代理与固定预算下的 paired benchmark 给出。